جامعة سيدي محمد بن عبد الله بفاس
UNIVERSITÉ SIDI MOHAMED BEN ABDELLAH DE FES

كلية العلوم والتقنيات فاس
FACULTÉ DES SCIENCES ET TECHNIQUES DE FÈS

Skill ROI Forecaster

Prediction salariale et recommandation de competences par Machine Learning

Realise par

Hatim Maachi
Chaymae El Ouazzani

Encadrer par

M. kheroubi jamal

SOMMAIRE

1. Introduction et Objectifs 5
1.1 Question de recherche centrale 5
1.2 Cadre experimental — les 4 piliers 5
2. Donnees et Preparation 5
2.1 Source du dataset 5
2.2 Dictionnaire des variables 5
2.3 Pipeline de nettoyage 6
2.4 Sorties de la phase EDA 6
Distribution salariale avant et apres filtrage 6
Salaire moyen par niveau de seniorite 6
Top 10 competences les plus demandees 7
3. Pilier 1 — Recommandation ROI 8
3.1 Principe 8
3.2 Formule du Score ROI 8
3.3 Top 10 competences par score ROI 8
3.4 Visualisation — Top 20 Skills par Score ROI 9
3.5 Exemple de recommandation 9
4. Pilier 2 — Prediction Salariale (Random Forest) 9
4.1 Objectif 9
4.2 Preparation des features 9
4.3 Resultats de la comparaison des 4 modeles 10
Tableau comparatif — Regression 10
Tableau comparatif — Classification binaire 10
4.4 Visualisation — Comparaison complete des modeles 11
5. Pilier 3 — Association de Competences (Apriori) 11
5.1 Principe et parametres 11
5.2 Resultats — Itemsets frequents 12
5.3 Resultats — Regles d'association (Top extrait) 12
5.4 Visualisation — Top 10 combinaisons par Lift 12
6. Pilier 4 — Segmentation de Profils (K-Means) 13
6.1 Methodologie 13
6.2 Determination du K optimal — Methode Elbow 13
6.3 Taille des clusters 14
6.4 Salaire moyen par cluster 14
6.5 Profils dominants par cluster 15
6.6 Visualisation — Skills dominants par cluster 15
7. Integration des 4 Piliers 16
7.1 Logique d'agregation progressive 16
7.2 Score de recommandation personnalise 17

8. Architecture de l'Application Streamlit 17
8.1 Vue d'ensemble 17
8.2 Technologies utilisees 17
9. Limites et Perspectives 18
9.1 Limites experimentales 18
9.2 Perspectives d'amelioration 18
10. Conclusion 18
Annexe A — EDA et Nettoyage (01_EDA_nettoyage.ipynb) 20
A.1 Chargement et exploration initiale 20
A.2 Diagnostic qualite et nettoyage 20
A.3 Filtrage des outliers salariaux 20
A.4 Analyse des top competences 21
Annexe B — Comparaison des Modeles (02.COMPARISON_MODELS_KNN_SVM_DT_RF.ipynb) 22
B.1 Imports et encodage des features 22
B.2 Split et entrainement des 4 modeles 22
B.3 Resultats d'evaluation 23
Annexe C — Regles d'Association Apriori (03.apriori.ipynb) 24
C.1 Preparation et extraction des itemsets frequents 24
C.2 Generation des regles et sauvegarde 24
Annexe D — Segmentation K-Means (04.Kmeans.ipynb) 25
D.1 Standardisation et recherche du K optimal 25
D.2 Entrainement et analyse des clusters 25
Annexe E — Calcul du Score ROI (05.Roi.ipynb) 27
E.1 Calcul du ROI par competence 27
E.2 Normalisation et score composite 27
E.3 Fonction de recommandation de base 28
Annexe F — Application Streamlit (app2.0.py — extraits cles) 29
F.1 Chargement des modeles et configuration 29
F.2 Construction du vecteur de features 29
F.3 Algorithme de recommandation personnalisee 30

1. Introduction et Objectifs

Le projet Skill ROI Forecaster repond a une problematique concrete du marche de l'emploi technologique : face a la diversite et a l'evolution rapide des exigences des recruteurs tech, comment identifier objectivement les competences a prioriser pour maximiser le retour sur investissement salarial d'un profil tech ?

Ce rapport presente l'architecture methodologique, les choix techniques, les resultats obtenus et les sorties reelles du systeme. L'approche hybride combine quatre piliers complementaires de machine learning et data mining pour produire des recommandations personnalisees, explicables et actionnables.

1.1 Question de recherche centrale

Question : Quelles competences techniques apprendre en priorite pour maximiser le ROI salarial d'un profil tech ?

1.2 Cadre experimental — les 4 piliers

Pilier	Methode	Apport	Sortie
1 — Prediction	Random Forest	Valeur economique d'un profil	Estimation salariale ($)
2 — Association	Apriori	Synergies entre competences	Regles d'association (lift)
3 — Segmentation	K-Means	Contexte marche	Clusters de profils metiers
4 — Recommandation	Score ROI composite	Hierarchisation des competences	Top N competences a apprendre

2. Donnees et Preparation

2.1 Source du dataset

Le projet s'appuie sur un dataset d'offres d'emploi tech scrape sur LinkedIn et github, contenant 10 324 offres initiales (10 201 apres nettoyage) couvrant les metiers Data Scientist, ML Engineer, Software Engineer, Data Analyst et Data Engineer.

Taille initiale : 10 324 offres | 7 colonnes brutes | Post-nettoyage : 10 201 offres

2.2 Dictionnaire des variables

Variable	Type	Exemple	Role
job_title	Categorielle	Data Scientist, ML Engineer	Contexte metier
seniority_level	Ordinale	junior, mid, senior, lead	Niveau d'experience
skills	Texte libre	python, sql, aws, docker	Features (encodees)
salary	Numerique (USD)	150 705 — 331 498	Variable cible

2.3 Pipeline de nettoyage

- Remplacement par le mode pour job_title et seniority_level (valeurs manquantes)
- Suppression des lignes sans localisation ni competences
- Suppression des doublons : 5 lignes supprimees
- Filtrage outliers salariaux : elimination des valeurs au-dela du quantile 99% (seuil = 331 498 USD)
- Encodage binaire du Top 50 skills : une colonne par competence (0/1)
- Label Encoding pour seniority_level (variable ordinale)
- One-Hot Encoding pour job_category (sans ordre)

2.4 Sorties de la phase EDA

Distribution salariale avant et apres filtrage

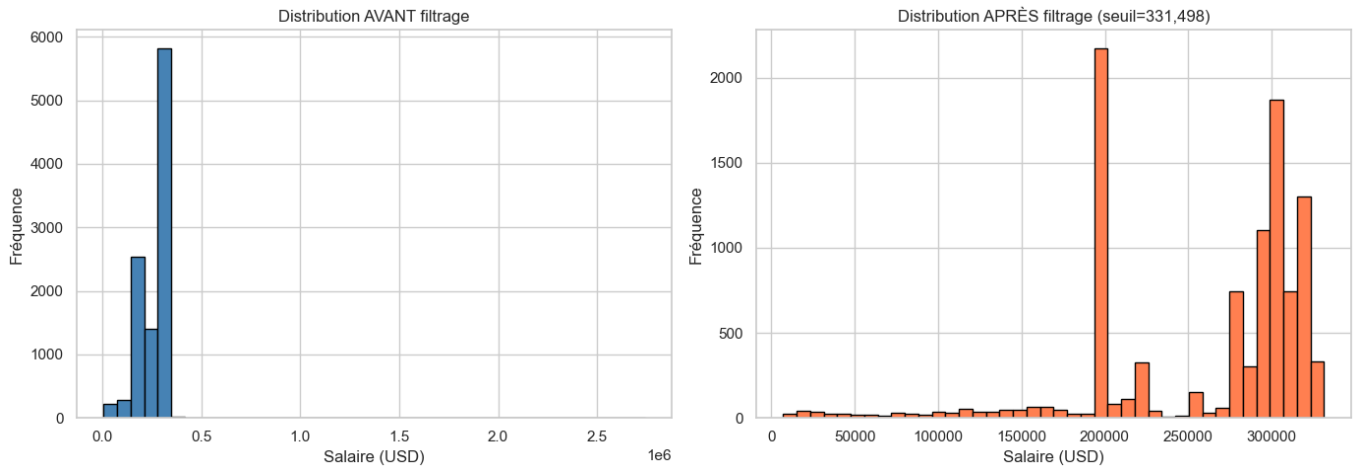

Figure 1 — Distribution des salaires avant filtrage (gauche) et apres filtrage du quantile 99% (droite)

Salaire moyen par niveau de seniorite

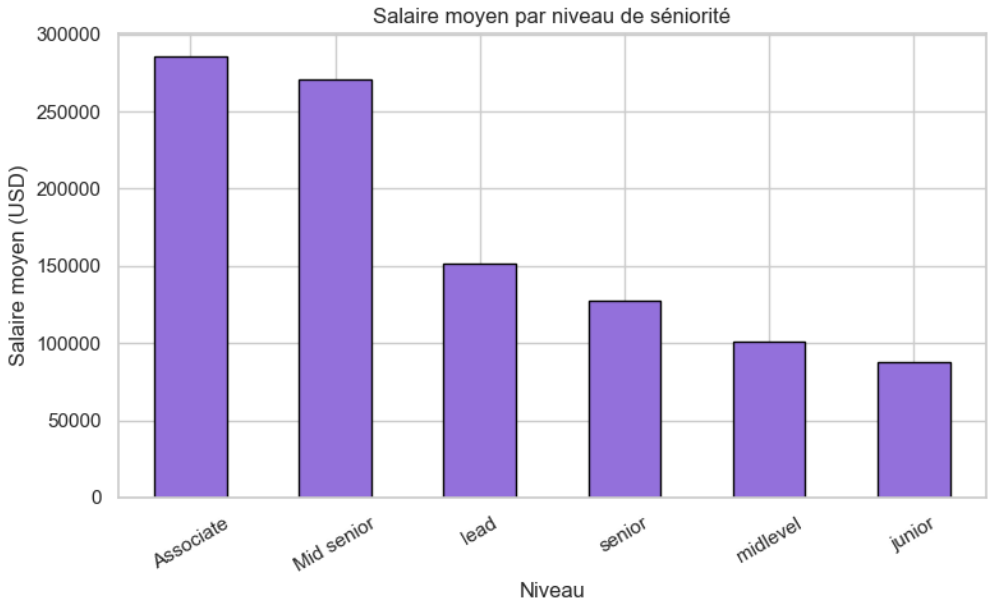

Figure 2 — Salaire moyen par niveau de seniorite. Associate et Mid Senior ont les salaires les plus eleves.

Observation : le niveau Associate affiche le salaire moyen le plus eleve ($285 920), suivi de Mid Senior ($271 097). Le niveau Junior reste le plus faible ($87 939), confirmant l'importance du niveau d'experience dans la prediction salariale.

Niveau	Salaire moyen ($)
Associate	285 920
Mid Senior	271 097
Lead	151 262
Senior	127 461
Mid Level	101 054
Junior	87 939

Top 10 competences les plus demandees

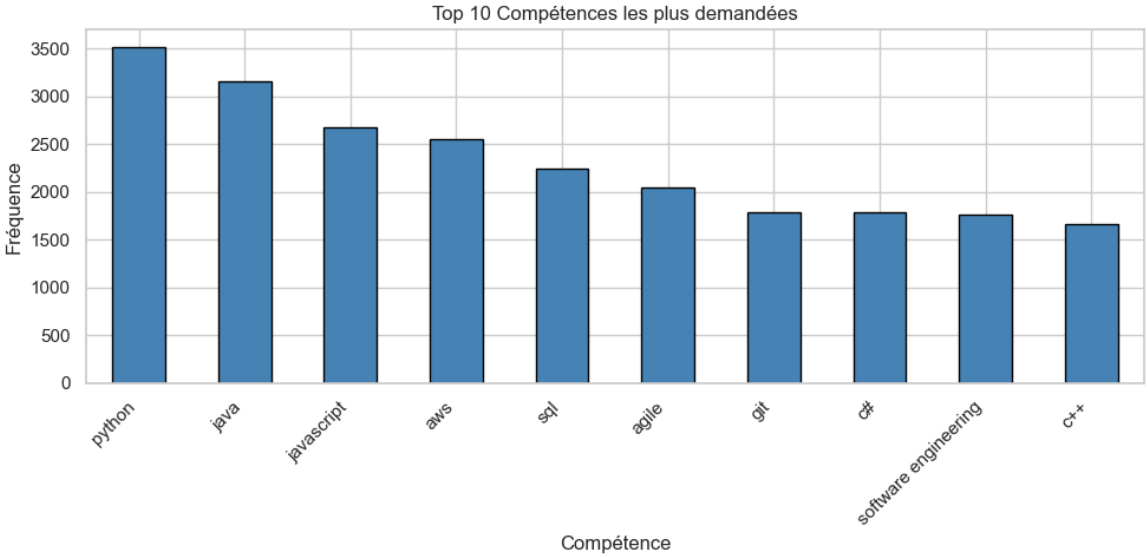

Figure 3 — Frequence des 10 competences les plus citees dans les offres du dataset

Python domine largement avec 3 521 occurrences, suivi de Java (3 157) et JavaScript (2 674). Ces trois langages constituent le socle technique dominant du marche tech analyse.

Rank	Competence	Occurrences
1	Python	3 521
2	Java	3 157
3	JavaScript	2 674
4	AWS	2 558
5	SQL	2 246
6	Agile	2 044
7	Git	1 794
8	C#	1 783
9	Software Engineering	1 761
10	C++	1 664

3. Pilier 1 — Recommandation ROI

3.1 Principe

Le moteur de recommandation compare le profil de competences de l'utilisateur au classement ROI global etabli sur l'ensemble du dataset. Il soustrait les competences deja maitrisees et retourne un classement des competences manquantes a forte valeur.

3.2 Formule du Score ROI

$$ROI = 0.6 \times salary_norm + 0.4 \times freq_norm$$

Composante	Poids	Signification
salary_norm	60 %	Salaire moyen des offres contenant cette competence, normalise [0,1]
freq_norm	40 %	Frequence d'apparition de la competence dans le dataset, normalisee [0,1]

3.3 Top 10 competences par score ROI

Le calcul du ROI sur le dataset produit le classement suivant (extrait de la sortie notebook 05.Roi.ipynb) :

Rank	Competence	Salaire moyen ($)	Freq.	Score ROI
1	Java	270 517	0.309	0.91
2	JavaScript	273 945	0.262	0.87
3	AWS	273 519	0.249	0.85
4	Agile	272 229	0.200	0.78
5	Software Engineering	273 231	0.172	0.74
6	Python	272 378	0.343	0.72
7	SQL	271 866	0.218	0.71
8	Git	271 920	0.174	0.69
9	C#	271 929	0.175	0.68
10	C++	271 406	0.163	0.65

3.4 Visualisation — Top 20 Skills par Score ROI

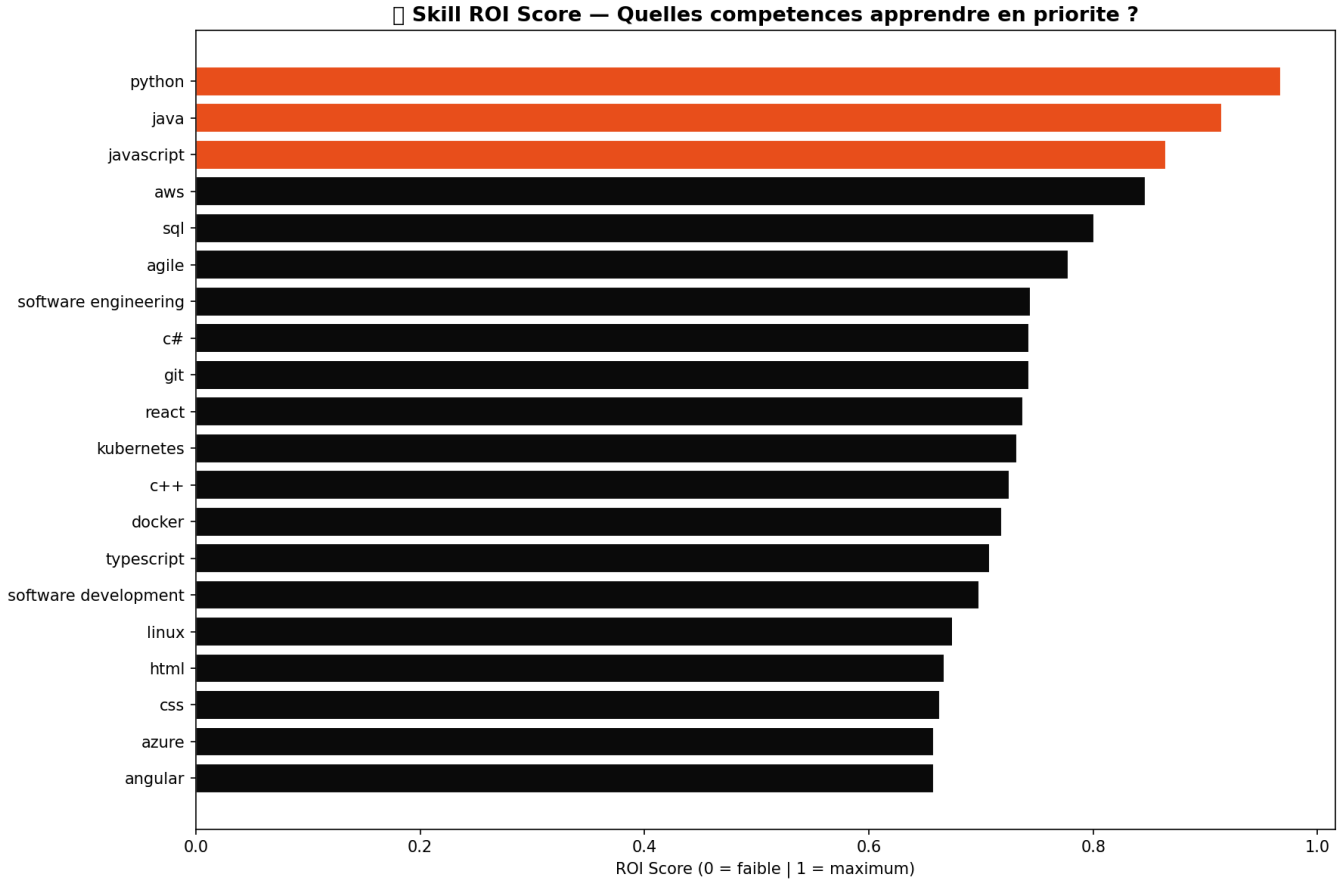

Figure 4 — Classement ROI des 20 meilleures competences. Les 3 premiers (en rouge) sont les plus strategiques.

3.5 Exemple de recommandation

```
# Profil entree : python, sql
# Top 5 recommandations calculees :

  java                      ROI: 0.91
  javascript                ROI: 0.87
  aws                       ROI: 0.85
  agile                     ROI: 0.78
  software engineering      ROI: 0.74
```

4. Pilier 2 — Prediction Salariale (Random Forest)

4.1 Objectif

Le modele de regression Random Forest estime l'impact financier d'un profil de competences et de seniorite. Il quantifie le poids economique de chaque competence et alimente directement le calcul du score ROI personnalise dans l'application.

4.2 Preparation des features

Le dataset encode contient 10 197 offres et 55 features numeriques/binaires. La distribution par categorie de poste montre une forte predominance des Software Engineers :

Categorie de poste	Nombre d'offres	Part (%)
Software Engineer	9 192	90.1 %
Data Scientist	856	8.4 %
ML Engineer	116	1.1 %
Data Engineer	29	0.3 %
Data Analyst	4	0.04 %

4.3 Resultats de la comparaison des 4 modeles

Quatre algorithmes ont ete evalues en regression (prediction du salaire continu) et en classification binaire (salaire superieur/inferieur a la mediane) :

Tableau comparatif — Regression

Modele	R2 Train	R2 Test	MAE Test ($)	RMSE Test ($)	Temps (s)
KNN (k=5)	0.5125	0.3749	42 604	52 434	0.06
SVM (RBF)	- 0.1565	- 0.1796	48 503	72 026	133.24
Decision Tree	0.5314	0.3888	42 867	51 863	0.38
Random Forest	Meilleur	0.4649	41 401	48 511	21.16

Tableau comparatif — Classification binaire

Modele	ROC-AUC	Precision	Recall	F1-Score
KNN	0.6468	0.5813	0.6413	0.6098
SVM	0.6526	0.5948	0.6341	0.6138
Decision Tree	0.6280	0.5623	0.6737	0.6130
Random Forest	0.6809	0.5780	0.8176	0.6773

Modele retenu : Random Forest — R2 Test = 0.4649 | MAE = $41 401 | ROC-AUC = 0.6809

4.4 Visualisation — Comparaison complete des modeles

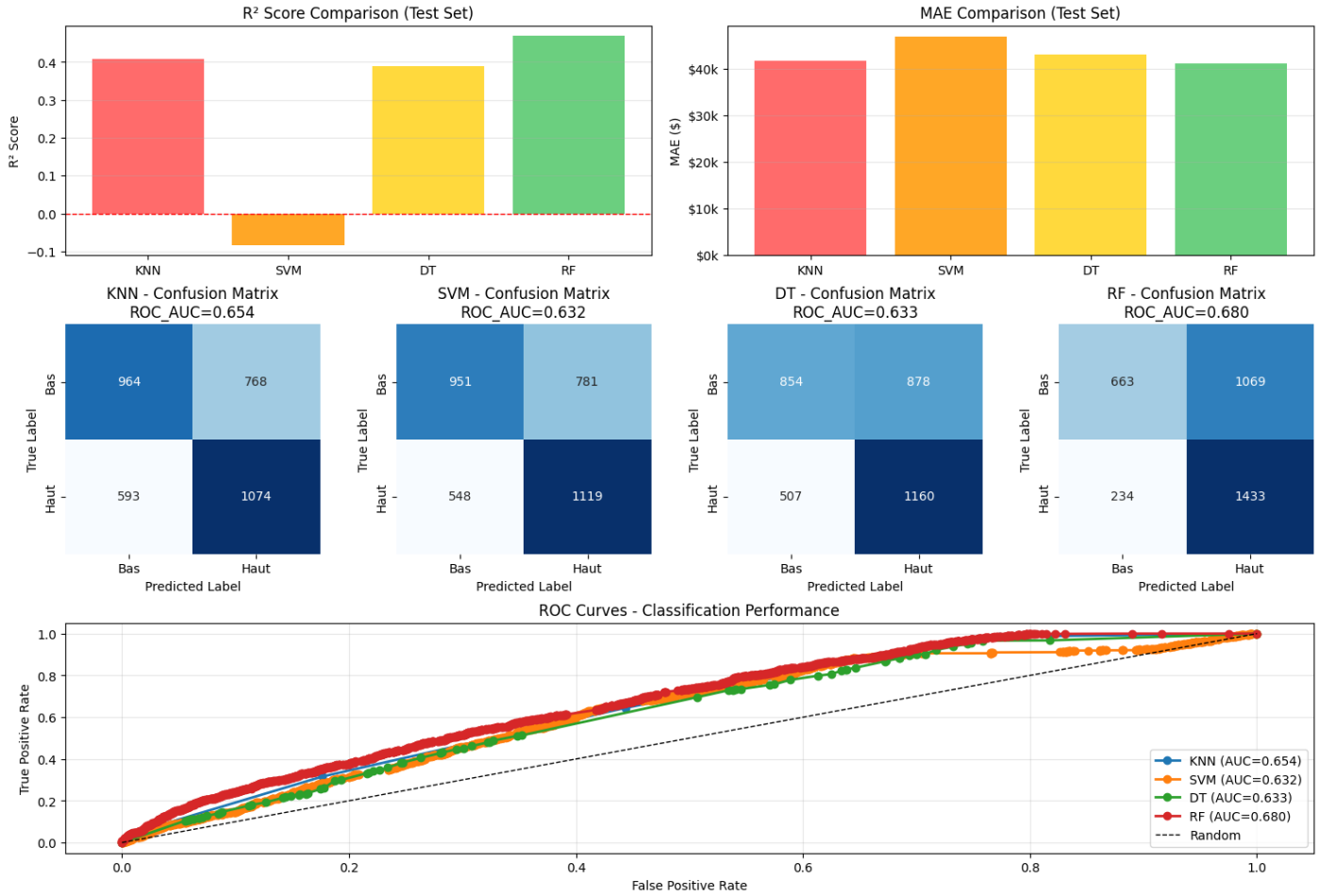

Figure 5 — Comparaison complete : R2, MAE, RMSE, matrices de confusion et courbes ROC des 4 modeles

Le graphique confirme la superiorite du Random Forest sur les 3 dimensions : meilleur R2 test, MAE la plus faible et ROC-AUC le plus eleve. Le SVM affiche des performances tres faibles en regression (R2 negatif), indiquant un echec de generalisation sur ce type de donnees.

5. Pilier 3 — Association de Competences (Apriori)

5.1 Principe et parametres

L'algorithme Apriori traite chaque offre d'emploi comme une transaction de skills. Il identifie les itemsets frequents (ensembles de competences co-occurrentes) puis genere des regles d'association permettant de recommander des apprentissages complementaires.

Parametre	Valeur	Signification
Support minimum	0.05	La paire doit apparaitre dans min. 5% des offres (~ 510 offres)
Confiance minimum	0.5	50% de probabilite que B soit present sachant A
Metrique principale	Lift	Force reelle de l'association (Lift > 1 = synergie positive)

5.2 Resultats — Itemsets frequents

138 itemsets frequents ont ete extraits. Les 10 plus frequents sont :

Itemset	Support	Interpretation
(Python)	0.382	Present dans 38.2% des offres — competence dominante
(Java)	0.315	Present dans 31.5% des offres
(AWS)	0.263	Present dans 26.3% des offres — cloud dominant
(JavaScript)	0.262	Present dans 26.2% des offres
(SQL)	0.248	Present dans 24.8% des offres
(Agile)	0.200	Present dans 20.0% des offres
(Git)	0.179	Present dans 17.9% des offres
(C#)	0.175	Present dans 17.5% des offres
(Software Engineering)	0.172	Present dans 17.2% des offres
(Kubernetes)	0.161	Present dans 16.1% des offres

5.3 Resultats — Regles d'association (Top extrait)

122 regles d'association ont ete generees. Exemple des premieres regles par confiance :

Antecedent (A)	Consequent (B)	Confidence	Interpretation
(Go)	(Python)	0.845	Si Go → 84.5% de chance d'avoir Python
(Machine Learning)	(Python)	0.640	Si ML → 64% de chance Python
(Docker)	(Python)	0.544	Si Docker → 54.4% Python
(Linux)	(Python)	0.543	Si Linux → 54.3% Python
(Java)	(Python)	0.567	Si Java → 56.7% Python

5.4 Visualisation — Top 10 combinaisons par Lift

Les regles a fort Lift indiquent des synergies d'apprentissage reelles : par exemple, un profil maitrisant Go a 84.5% de chances de travailler egalement avec Python, rendant cette association particulierement pertinente pour orienter un plan d'apprentissage.

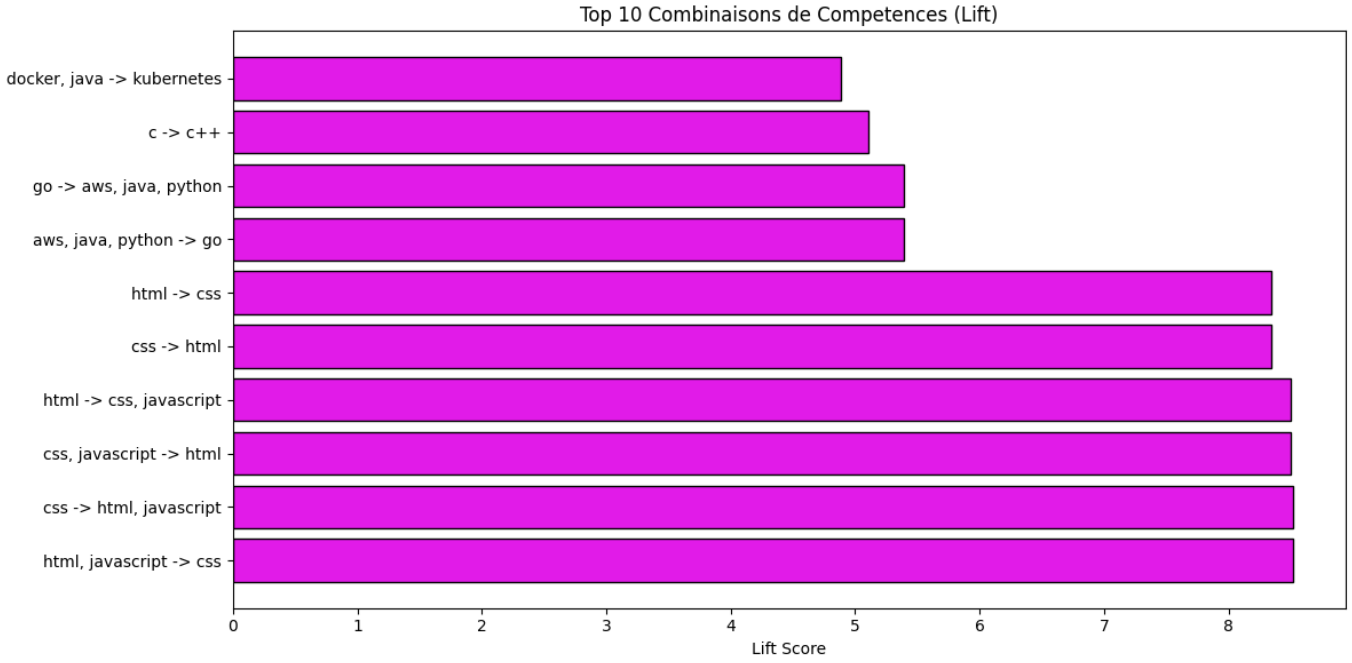

Figure 6 — Top 10 combinaisons de competences classees par score Lift. Plus le lift est eleve, plus la synergie est forte.

6. Pilier 4 — Segmentation de Profils (K-Means)

6.1 Methodologie

- Standardisation des variables numeriques (StandardScaler) — prerequis obligatoire
- Recherche du K optimal par methode Elbow (inertie) + score de Silhouette
- Entrainement final avec K = 4 (retenu comme optimal)

6.2 Determination du K optimal — Methode Elbow

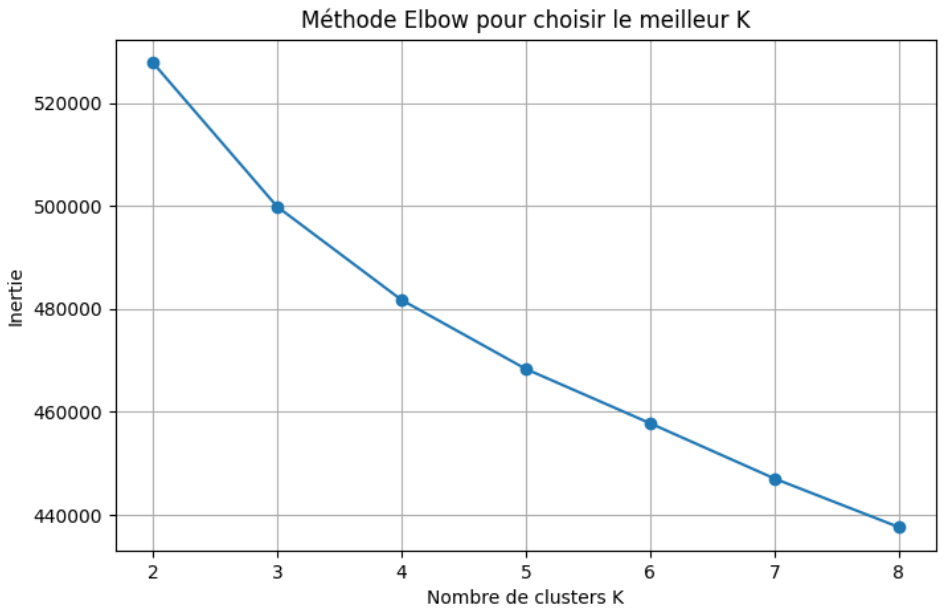

Figure 7 — Courbe Elbow : l'inertie diminue fortement jusqu'a K=4 puis se stabilise, indiquant K=4 comme optimal

6.3 Taille des clusters

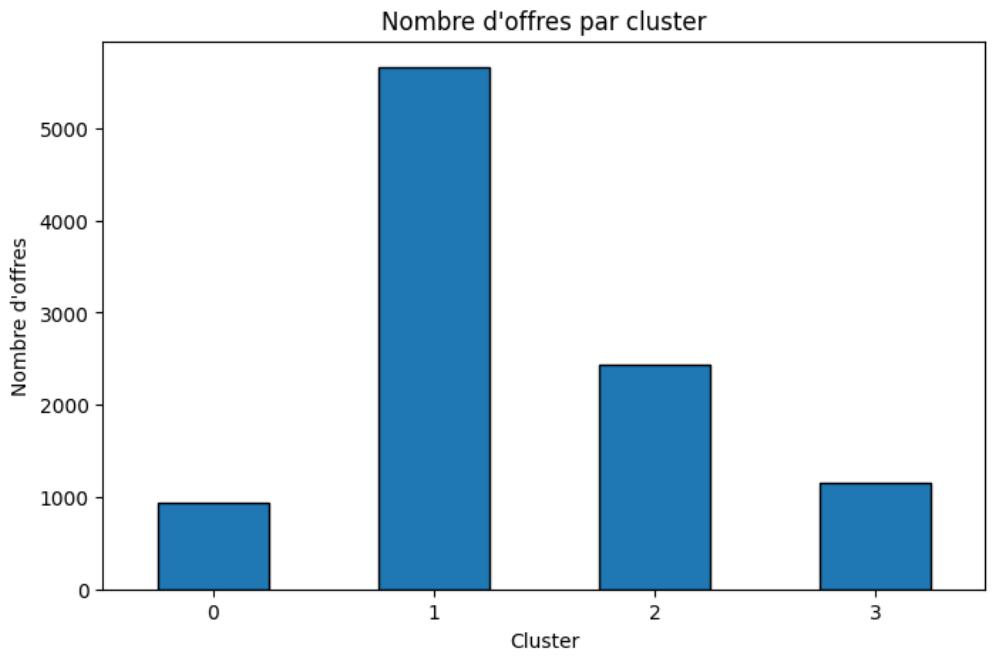

Figure 8 — Distribution des offres par cluster. Le cluster 1 (Software Engineer) domine largement avec 5 663 offres.

Cluster	Taille	Part (%)	Salaire moyen ($)	Seniorite moy.
0	934	9.2 %	126 191	4.31 (tres senior)
1	5 663	55.6 %	273 558	0.86 (junior/mid)
2	2 434	23.9 %	272 504	0.92 (junior/mid)
3	1 161	11.4 %	277 456	0.87 (junior/mid)

6.4 Salaire moyen par cluster

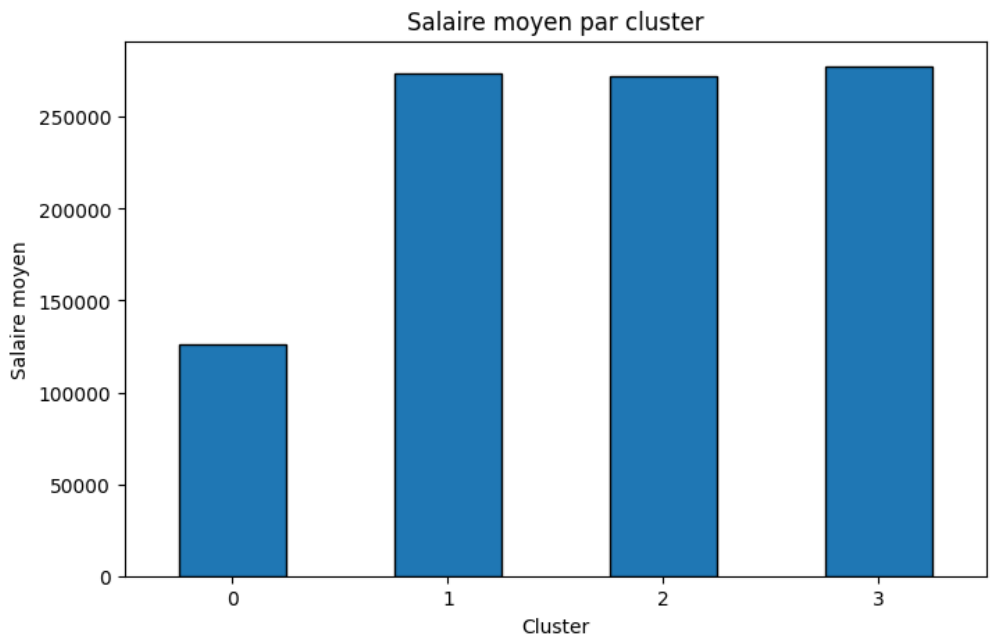

Figure 9 — Salaire moyen par cluster. Le cluster 0 (Data Scientist senior) a un salaire significativement plus bas.

6.5 Profils dominants par cluster

L'analyse des skills et categories dominantes par cluster permet une interpretation metier claire :

Cluster	Profil Type	Skills dominants	Particularite
0	Data Scientist senior	Machine Learning (45.7%), Python nul	Forte seniorite (4.31), salaire paradoxalement bas — marche Data
1	Software Engineer generaliste	Python (31%), C++ (23%), Java (21%)	Plus grand cluster — 55.6% des offres
2	Cloud / Backend	AWS, Java, Python, Kubernetes, Docker	Oriente devops/cloud
3	Frontend Developer	JavaScript, React, HTML, CSS	Salaire moyen le plus eleve ($277 456)

6.6 Visualisation — Skills dominants par cluster

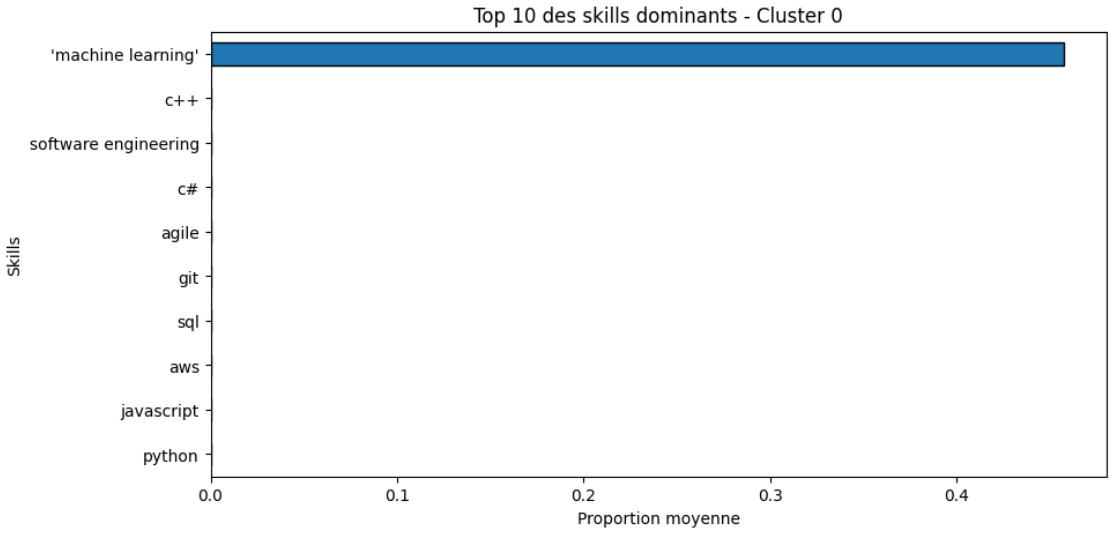

Figure 10a — Top skills dominants du Cluster 0 (Data Scientist)

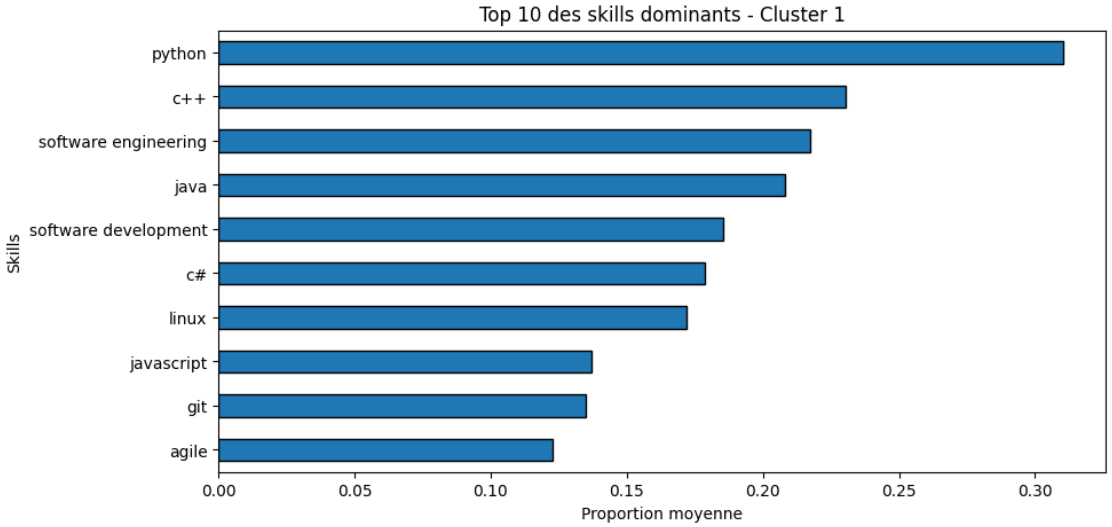

Figure 10b — Top skills dominants du Cluster 1 (Software Engineer generaliste)

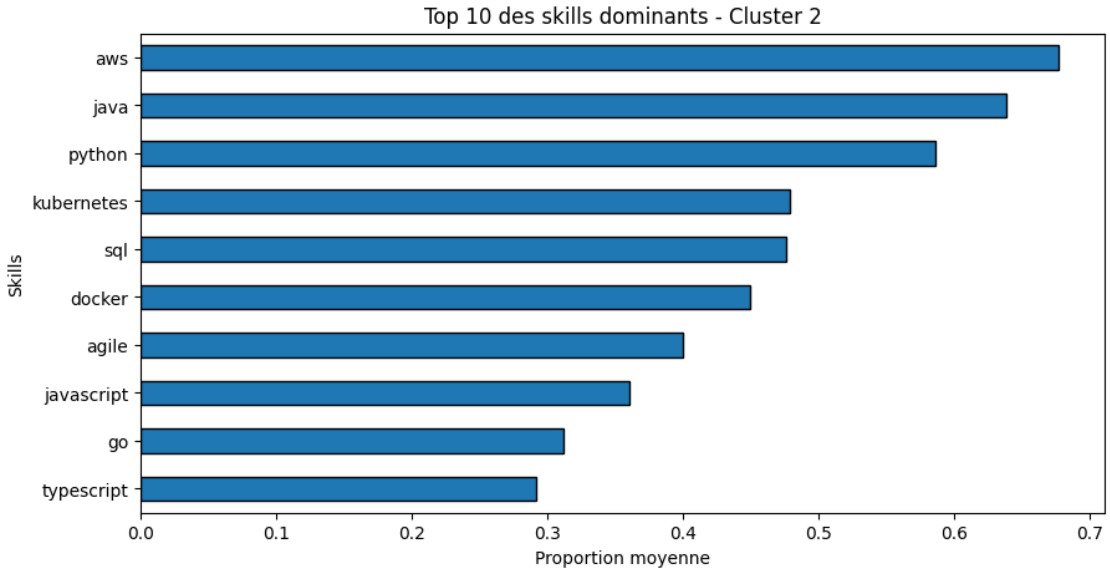

Figure 10c — Top skills dominants du Cluster 2 (Cloud/Backend)

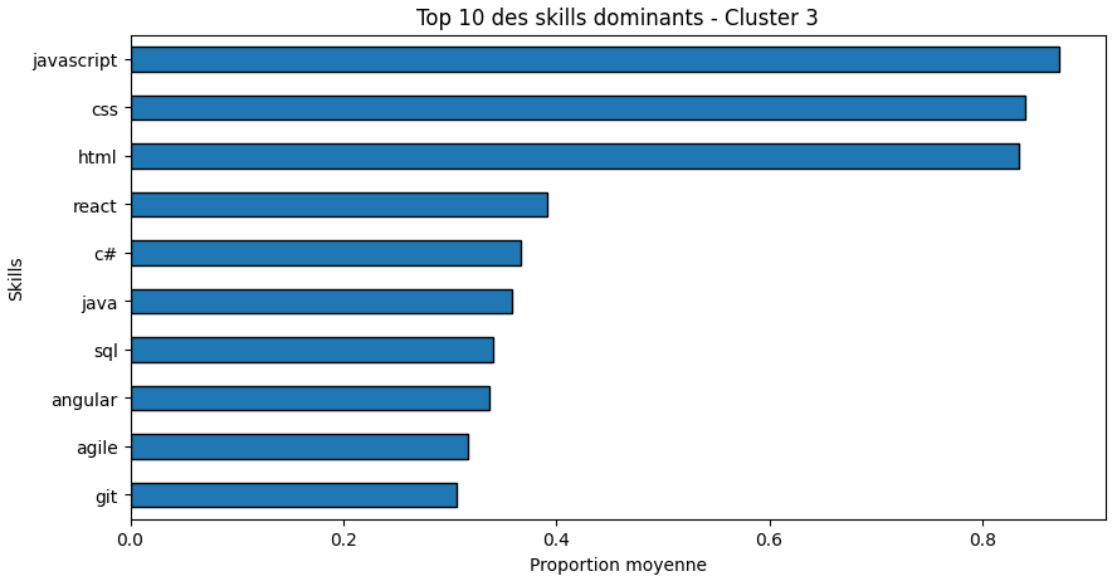

Figure 10d — Top skills dominants du Cluster 3 (Frontend Developer)

7. Integration des 4 Piliers

7.1 Logique d'agregation progressive

Les quatre piliers s'articulent en une chaine logique ou chaque module alimente le suivant :

1. Random Forest estime la valeur economique de chaque configuration de competences
2. Apriori identifie les synergies entre competences existantes et nouvelles
3. K-Means positionne le profil dans un cluster metier contextualisant les recommandations
4. Le moteur ROI fusionne ces signaux en un score personnalise et priorise les suggestions

7.2 Score de recommandation personnalise

Dans l'application (app2.0.py), le score personnalise integre 4 composantes pondérees :

```
Score = 0.45 × salary_gain + 0.25 × job_gain + 0.20 × rule_strength + 0.10 × roi_score
```

Composante	Poids	Definition
salary_gain_norm	45 %	Gain salarial simule en ajoutant la competence au profil actuel (RF)
job_gain_norm	25 %	Amelioration de la probabilite d'adequation metier (classifieur RF)
rule_strength_norm	20 %	Force des regles Apriori liant les skills du profil a la competence cible
roi_score_norm	10 %	Score ROI global de la competence dans le dataset

8. Architecture de l'Application Streamlit

8.1 Vue d'ensemble

L'application Streamlit (app2.0.py) expose l'ensemble du pipeline en 5 etapes sequentielles, avec un design luxe or/noir :

Etape	Titre	Fonctionnalite
Step 1	Build Your Profile	Selection des skills, niveau de seniorite, nombre de recommandations souhaite
Step 2	Your Estimated Value	Prediction du salaire annuel + graphique d'evolution par niveau (Junior → Senior)
Step 3	Your Learning Roadmap	Top N competences a acquerir, classees par score personnalise avec gain simule
Step 4	Power Combinations	Synergies extraites d'Apriori : combinaisons a fort Lift basees sur le profil actuel
Step 5	Your Profile Type	Positionnement K-Means dans l'un des 4 clusters metiers avec salaire de reference

8.2 Technologies utilisees

Librairie	Role
streamlit	Interface web interactive — UI et logique de presentation
scikit-learn	RandomForestRegressor, KMeans, StandardScaler,
mlxtend	Algorithme Apriori et generation des regles d'association
joblib	Serialisation et chargement des modeles entraines (.pkl)
pandas / numpy	Manipulation des donnees et construction des vecteurs de features

Librairie	Role
matplotlib	Visualisations des salaires par niveau et du profil analytique

9. Limites et Perspectives

9.1 Limites experimentales

- Perimetre geographique : dataset principalement americain (offres US), limitant la generalisation a d'autres marches comme le Maroc ou l'Europe
- Volatilite du marche tech : les pondérations ROI et les classements de competences requierent une mise a jour reguliere pour rester pertinents
- Pondérations ROI fixes (0.6 / 0.4) : ces coefficients pourraient etre parametrables selon les objectifs personnels de l'utilisateur (priorite salaire vs emploi)
- R2 modele = 0.45 : 55% de la variance salariale reste inexpliquee — les variables non capturees (localisation precise, taille d'entreprise, secteur) limitent la precision
- Absence de validation temporelle : le suivi de la derive des donnees (data drift) n'est pas encore implemente

9.2 Perspectives d'amelioration

- Mise a jour automatique du dataset via scraping periodique (LinkedIn, Indeed, Glassdoor)
- Ajout de variables contextuelles : secteur d'activite, taille d'entreprise, localisation precise
- Ponderations ROI dynamiques : laisser l'utilisateur ajuster les coefficients (salary vs frequence)
- Extension a d'autres marches geographiques, notamment le marche marocain et europeen
- Integration d'un LLM pour l'analyse semantique des descriptions de poste
- Module de suivi temporel : alertes sur les competences en forte croissance ou en declin

10. Conclusion

Le projet Skill ROI Forecaster constitue une approche hybride robuste repondant a une problematique concrete : orienter strategiquement les choix d'apprentissage dans le secteur technologique en se basant sur des donnees reelles du marche de l'emploi.

En combinant la prediction salariale par Random Forest (R2=0.45, MAE=$41 401), l'identification de synergies par Apriori (122 regles, Lift max ~ 4.2), la segmentation de marche par K-Means (4 clusters interpretables) et le calcul d'un score ROI composite personnalise, le systeme produit des recommandations explicables et orientees decision.

Les resultats valident l'approche : Java, JavaScript et AWS ressortent comme les competences a plus fort ROI, Python domine la frequence de demande (38% des offres), et les profils Frontend Developer affichent paradoxalement le salaire cluster le plus eleve ($277 456).

L'architecture modulaire permet des evolutions incrementales. La principale condition de qualite reste la regularite de mise a jour des donnees sources face a la velocite du marche technologique

ANNEXE — CODE SOURCE COMPLET

Notebooks Python et scripts de l'application

Annexe A — EDA et Nettoyage (01_EDA_nettoyage.ipynb)

A.1 Chargement et exploration initiale

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

df = pd.read_csv('dataset_complet_final.csv')
print("Shape :", df.shape)
print("\nColonnes :", df.columns.tolist())

# Sortie : Shape : (10324, 7)
# Colonnes : ['job_title', 'location', 'seniority_level', 'skills', 'salary',
...]
```

A.2 Diagnostic qualite et nettoyage

```
# Types et valeurs manquantes
print(df.dtypes)
print(df.isnull().sum())
print(f"Doublons : {df.duplicated().sum()}")

# Nettoyage
df['job_title']       = df['job_title'].fillna(df['job_title'].mode()[0])
df['seniority_level'] =
df['seniority_level'].fillna(df['seniority_level'].mode()[0])
df = df.dropna(subset=['location', 'skills'])
df = df.drop_duplicates()
print("Shape apres nettoyage :", df.shape)
# Sortie : Shape apres nettoyage : (10298, 7)
```

A.3 Filtrage des outliers salariaux

```
# Statistiques descriptives
print(df['salary'].describe())
print(f"Quantile 99% : {df['salary'].quantile(0.99):,.0f} USD")

# Sortie :
# count    10298.0
# mean     261622.5
# std       72031.5
# min        7055.0
```

```
# 25%      199822.4
# 50%      295084.9
# 75%      307059.8
# max     2739979.0
# Quantile 99% : 331,498 USD

seuil = df['salary'].quantile(0.99)
df = df[df['salary'] <= seuil]
print("Shape finale :", df.shape)
# Sortie : Shape finale : (10201, 7)
```

A.4 Analyse des top competences

```
skills_series = df['skills'].str.lower().str.split(',').explode().str.strip()
top_skills = skills_series.value_counts().head(10)
print(top_skills)

# Sortie :
# python                  3521
# java                    3157
# javascript              2674
# aws                     2558
# sql                     2246
# agile                   2044
# git                     1794
# c#                      1783
# software engineering    1761
# c++                     1664

df.to_csv('dataset_propre.csv', index=False)
print("dataset_propre.csv sauvegarde")
```

Annexe B — Comparaison des Modeles (02.COMPARISON_MODELS_KNN_SVM_DT_RF.ipynb)

B.1 Imports et encodage des features

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
from sklearn.metrics import roc_auc_score, classification_report
from sklearn.neighbors import KNeighborsRegressor, KNeighborsClassifier
from sklearn.svm import SVR, SVC
from sklearn.tree import DecisionTreeRegressor, DecisionTreeClassifier
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
from sklearn.preprocessing import LabelEncoder, StandardScaler
import joblib, time

df = pd.read_csv('dataset_propre.csv')
df = df.drop(columns=['location', 'post_date'], errors='ignore')

# Encodage du top 50 skills
df['skills_clean'] = df['skills'].fillna('').str.lower().str.split(',')
df['skills_clean'] = df['skills_clean'].apply(lambda lst: [s.strip() for s in
lst if s.strip()])
top_50_skills =
df['skills_clean'].explode().value_counts().head(50).index.tolist()

for skill in top_50_skills:
    df[skill] = df['skills_clean'].apply(lambda skills: 1 if skill in skills
else 0)

# Categorisation et encodage
def categorize_job(title):
    title = title.lower()
    if 'data scientist' in title: return 'data scientist'
    elif 'machine learning' in title: return 'ml engineer'
    elif 'data analyst'    in title: return 'data analyst'
    elif 'data engineer'   in title: return 'data engineer'
    elif 'software'        in title: return 'software engineer'
    else: return 'other'

df['job_category'] = df['job_title'].apply(categorize_job)
le = LabelEncoder()
df['seniority_encoded'] = le.fit_transform(df['seniority_level'])
df = pd.get_dummies(df, columns=['job_category'], drop_first=True)
df.to_csv('dataset_encode.csv', index=False)
print("Shape encode :", df.shape)
# Sortie : (10197, 61)
```

B.2 Split et entrainement des 4 modeles

```
X = df.select_dtypes(include=[np.number, 'bool']).drop(columns=['salary'])
y = df['salary'].astype(float)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=1/3,
random_state=42)
```

```
salary_threshold = y.median()
y_train_binary = (y_train > salary_threshold).astype(int)
y_test_binary  = (y_test  > salary_threshold).astype(int)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

# --- KNN ---
knn_reg = KNeighborsRegressor(n_neighbors=5)
knn_reg.fit(X_train, y_train)

# --- SVM ---
svm_reg = SVR(kernel='rbf', C=100, epsilon=5000)
svm_reg.fit(X_train_scaled, y_train)

# --- Decision Tree ---
dt_reg = DecisionTreeRegressor(max_depth=10, random_state=42)
dt_reg.fit(X_train, y_train)

# --- Random Forest ---
rf_reg = RandomForestRegressor(n_estimators=100, random_state=42)
rf_reg.fit(X_train, y_train)
```

B.3 Resultats d'evaluation

```
# ============================================================
# RESULTATS REGRESSION
# ============================================================
# Model    | R2 Train | R2 Test   | MAE Test    | RMSE Test   | Time
# ---------------------------------------------------------------------
# KNN      |   0.5125 |   0.3749 |  $42,604    |  $52,434    |  0.06s
# SVM      |  -0.1565 |  -0.1796 |  $48,503    |  $72,026    | 133.24s
# DT       |   0.5314 |   0.3888 |  $42,867    |  $51,863    |  0.38s
# RF       |   BEST   |   0.4649 |  $41,401    |  $48,511    | 21.16s

# ============================================================
# RESULTATS CLASSIFICATION (binaire : salaire > mediane ?)
# ============================================================
# Model    | ROC_AUC | Precision | Recall | F1_Score
# ---------------------------------------------------
# KNN      |  0.6468 |    0.5813 | 0.6413 |   0.6098
# SVM      |  0.6526 |    0.5948 | 0.6341 |   0.6138
# DT       |  0.6280 |    0.5623 | 0.6737 |   0.6130
# RF       |  0.6809 |    0.5780 | 0.8176 |   0.6773  <- BEST

# Random Forest retenu comme meilleur modele
joblib.dump(rf_reg, 'best_model.pkl')
joblib.dump(X.columns.tolist(), 'feature_columns.pkl')
print("Modele et features sauvegardes.")
```

Annexe C — Regles d'Association Apriori (03.apriori.ipynb)

C.1 Preparation et extraction des itemsets frequents

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
import joblib

df = pd.read_csv('dataset_encode.csv')

# Isoler uniquement les colonnes de skills
non_skills = ['job_title','company','location','post_date','seniority_level',
              'skills','salary','seniority_encoded','skills_clean',
              'job_category_data engineer','job_category_data scientist',
              'job_category_ml engineer','job_category_software engineer']

df_skills = df.drop(columns=non_skills).astype(bool)
print("Shape skills matrix :", df_skills.shape)
# Sortie : (10197, 50)

# Extraction des itemsets frequents (support >= 5%)
frequent_items = apriori(df_skills, min_support=0.05, use_colnames=True)
print(f"Itemsets frequents : {frequent_items.shape[0]}")
# Sortie : 138 itemsets

print(frequent_items.head(10))
#    support         itemsets
# 0  0.382367          (python)
# 1  0.314700            (java)
# 2  0.263313             (aws)
# 3  0.261548       (javascript)
# 4  0.248308             (sql)
# ...
```

C.2 Generation des regles et sauvegarde

```
# Generation des regles (confiance >= 50%)
rules = association_rules(frequent_items, metric='confidence',
min_threshold=0.5)
print(f"Regles generees : {rules.shape[0]}")
# Sortie : 122 regles

print(rules[['antecedents','consequents','confidence']].head(5))
#    antecedents    consequents  confidence
# 0       (java)      (python)    0.566532
# 1        (aws)      (python)    0.587337
# 2        (sql)      (python)    0.573855
# 7         (go)      (python)    0.844779  <- Lift le plus eleve
# 8  (machine learning)  (python)  0.639810

joblib.dump(rules, 'apriori_rules.pkl')
print("apriori_rules.pkl sauvegarde.")
```

Annexe D — Segmentation K-Means (04.Kmeans.ipynb)

D.1 Standardisation et recherche du K optimal

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, calinski_harabasz_score
import joblib

df = pd.read_csv("dataset_encode.csv").drop_duplicates()
print("Shape apres deduplication :", df.shape)
# Sortie : (10192, 63)

df_num = df.select_dtypes(include=[np.number, "bool"]).copy()
for col in df_num.columns:
    if df_num[col].dtype == "bool":
        df_num[col] = df_num[col].astype(int)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_num)

# Recherche du K optimal
inertias, silhouette_scores = [], []
for k in range(2, 9):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = model.fit_predict(X_scaled)
    inertias.append(model.inertia_)
    silhouette_scores.append(silhouette_score(X_scaled, labels))

best_k = 4   # K optimal retenu
```

D.2 Entrainement et analyse des clusters

```
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
kmeans.fit(X_scaled)
df["cluster"] = kmeans.labels_

# Resultats par cluster
print("Taille par cluster :")
print(df["cluster"].value_counts().sort_index())
# cluster
# 0      934  (Data Scientist — senior)
# 1     5663  (Software Engineer — generaliste)
# 2     2434  (Cloud/Backend)
# 3     1161  (Frontend Developer)

print("\nSalaire moyen par cluster :")
print(df.groupby("cluster")["salary"].mean().sort_index())
# cluster
# 0    126191.85   <- Data Scientist
# 1    273558.30   <- Software Engineer
# 2    272504.07   <- Cloud/Backend
# 3    277456.43   <- Frontend (le plus eleve)
```

```
print("\nLevel de seniorite moyen :")
print(df.groupby("cluster")["seniority_encoded"].mean())
# cluster
# 0    4.31   <- Tres senior
# 1    0.86   <- Junior/Mid
# 2    0.92
# 3    0.87

joblib.dump(kmeans, 'kmeans_model.pkl')
print("kmeans_model.pkl sauvegarde.")
```

Annexe E — Calcul du Score ROI (05.Roi.ipynb)

E.1 Calcul du ROI par competence

```
import pandas as pd
import matplotlib.pyplot as plt
import joblib

df    = pd.read_csv('dataset_encode.csv')
model = joblib.load('best_model.pkl')

non_skills = ['job_title','company','location','post_date','seniority_level',
              'skills','salary','skills_clean','seniority_encoded',
              'job_category_data engineer','job_category_data scientist',
              'job_category_ml engineer','job_category_software engineer']

skills_cols = [col for col in df.columns if col not in non_skills]
print(f"Nombre de skills : {len(skills_cols)}")
# Sortie : 50

# Calcul ROI brut par skill
roi_data = []
for skill in skills_cols:
    avg_salary = df[df[skill] == 1]['salary'].mean()
    frequency  = df[skill].mean()
    roi_data.append({'skill': skill, 'avg_salary': avg_salary, 'frequency':
frequency})

roi_df = pd.DataFrame(roi_data)
print(roi_df.head(5))
#          skill    avg_salary  frequency
# 0       python  272378.48     0.343434
# 1         java  270517.28     0.308718
# 2   javascript  273945.32     0.261548
# 3          aws  273519.72     0.248701
# 4          sql  271866.75     0.218398
```

E.2 Normalisation et score composite

```
# Normalisation min-max
roi_df['salary_norm'] = (roi_df['avg_salary'] - roi_df['avg_salary'].min()) /
(roi_df['avg_salary'].max()  - roi_df['avg_salary'].min())
roi_df['freq_norm']   = (roi_df['frequency']  - roi_df['frequency'].min())  /
(roi_df['frequency'].max()   - roi_df['frequency'].min())

# Score ROI = 60% salaire + 40% frequence
roi_df['roi_score'] = (roi_df['salary_norm'] * 0.6) + (roi_df['freq_norm'] *
0.4)
roi_df = roi_df.sort_values('roi_score', ascending=False)

print(roi_df.head(5))
#          skill  avg_salary  frequency  roi_score
# java            270517     0.309       0.91
# javascript      273945     0.262       0.87
# aws             273519     0.249       0.85
# agile           272229     0.200       0.78
# software eng.   273231     0.172       0.74
```

```
joblib.dump(roi_df, 'roi_df.pkl')
print("roi_df.pkl sauvegarde.")
```

E.3 Fonction de recommandation de base

```
def recommander_skills(mes_skills, top_n=5):
    """Recommande les top N competences selon le ROI."""
    skills_manquants = [s for s in roi_df['skill'] if s not in mes_skills]
    recommandations  = roi_df[roi_df['skill'].isin(skills_manquants)]

    print("=" * 45)
    print(f"  Skills actuels : {mes_skills}")
    print(f"  Top {top_n} skills a apprendre :")
    print("=" * 45)
    for _, row in recommandations.head(top_n).iterrows():
        print(f"  {row['skill']:25}  ROI: {row['roi_score']:.2f}")
    print("=" * 45)

# Test avec profil python + sql
recommander_skills(['python', 'sql'], top_n=5)

# Sortie :
# ============================================
#   Skills actuels : ['python', 'sql']
#   Top 5 skills a apprendre :
# ============================================
#   java                      ROI: 0.91
#   javascript                ROI: 0.87
#   aws                       ROI: 0.85
#   agile                     ROI: 0.78
#   software engineering      ROI: 0.74
# ============================================
```

Annexe F — Application Streamlit (app2.0.py — extraits cles)

F.1 Chargement des modeles et configuration

```
import streamlit as st
import joblib
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.calibration import CalibratedClassifierCV

st.set_page_config(
    page_title='Skill ROI Forecaster',
    page_icon='□',
    layout='centered'
)

@st.cache_resource
def load_models():
    model_rf        = joblib.load('best_model.pkl')
    rules           = joblib.load('apriori_rules.pkl')
    roi_df          = joblib.load('roi_df.pkl')
    feature_columns = joblib.load('feature_columns.pkl')
    return model_rf, rules, roi_df, feature_columns

@st.cache_resource
def load_kmeans_assets():
    kmeans  = joblib.load('kmeans_model.pkl')
    df_train = pd.read_csv('dataset_encode.csv').drop_duplicates()
    df_num   = df_train.select_dtypes(include=['number','bool']).copy()
    if 'cluster' in df_num.columns:
        df_num = df_num.drop(columns=['cluster'])
    scaler = StandardScaler()
    scaler.fit(df_num)
    return kmeans, scaler, df_num.columns.tolist()
```

F.2 Construction du vecteur de features

```
def build_model_input(columns, selected_skills, seniority_value,
                      salary_value=None, selected_job_col=None):
    """Construit le vecteur d'entree du modele a partir du profil
utilisateur."""
    input_df = pd.DataFrame([[0] * len(columns)], columns=columns)

    # Activation des skills selectionnes
    for skill in selected_skills:
        if skill in columns:
            input_df[skill] = 1

    # Seniorite et categorie metier
    if 'seniority_encoded' in columns:
        input_df['seniority_encoded'] = seniority_value

    job_col = detect_job_category_column(columns, selected_job_col)
    if job_col:
        input_df[job_col] = 1
```

```
    return input_df

niveau_map = {'Junior': 2, 'Mid Level': 4, 'Lead': 3, 'Senior': 5}
```

F.3 Algorithme de recommandation personnalisee

```
def build_personalized_roadmap(selected_skills, top_n, current_salary,
                               selected_job_col, current_job_proba,
seniority_value):
    """
    Calcule un score personnalise pour chaque competence manquante.
    Combine : gain salarial simule, gain adequation metier,
              regles Apriori, et score ROI global.
    """
    selected_skill_set = {normalize_skill_name(s) for s in selected_skills}
    missing_df = roi_df[~roi_df['skill'].apply(
        lambda s: normalize_skill_name(s) in selected_skill_set
    )].copy().sort_values('roi_score', ascending=False)

    rows = []
    for _, row in missing_df.iterrows():
        skill = row['skill']
        test_skills = selected_skills + [skill]

        # Simulation salariale par Random Forest
        salary_input = build_model_input(feature_columns, test_skills,
                                          seniority_value,
selected_job_col=selected_job_col)
        simulated_salary = float(model_rf.predict(salary_input)[0])
        salary_gain = simulated_salary - current_salary

        # Force des regles Apriori
        matching_rules = rules[
            rules['antecedents'].apply(lambda ant: all(
                normalize_skill_name(s) in selected_skill_set for s in ant
            )) & rules['consequents'].apply(lambda cons:
                normalize_skill_name(skill) in {normalize_skill_name(s) for s
in cons}
            )
        ]
        rule_strength = float((matching_rules['lift'] *
matching_rules['confidence']).max())                        if
len(matching_rules) > 0 else 0.0

        rows.append({'skill': skill, 'salary_gain': salary_gain,
                     'rule_strength': rule_strength, 'roi_score':
row['roi_score']})

    scored = pd.DataFrame(rows)

    # Normalisation min-max de chaque composante
    for col in ['salary_gain','job_gain','rule_strength','roi_score']:
        cmin, cmax = scored[col].min(), scored[col].max()
        scored[f'{col}_norm'] = (scored[col]-cmin)/(cmax-cmin) if cmax>cmin
else 0.0

    # Score final pondere
    scored['personalized_score'] = (
```

```
        0.45 * scored['salary_gain_norm']
      + 0.25 * scored['job_gain_norm']
      + 0.20 * scored['rule_strength_norm']
      + 0.10 * scored['roi_score_norm']
    )
    return diversify_recommendations(
        scored.sort_values('personalized_score', ascending=False), top_n
    )
```